Remote Backend – הסבר לעומק

למה בכלל צריך את זה?

כשמריצים `terraform apply` , Terraform שומר קובץ בשם **state** (`terraform.tfstate`) – זה "המוח" של Terraform: קובץ JSON שמתעד מה כבר נוצר בענן, עם אילו IDs ואילו attributes. בלי הקובץ הזה, Terraform לא יודע מה כבר קיים ומה צריך ליצור/לעדכן/למחוק.

הבעיה עם local state (ברירת המחדל – הקובץ יושב על הדיסק שלך בלבד):

- אם עובד שני בצוות מריץ `apply` – Terraform לא יודע על כך, וזה מוביל להתנגשויות (שני אנשים "חושבים" שהם היחידים שיוצרים/משנים infra)
- אם המחשב שלך נהרס או הקובץ נמחק – איבדת את התיעוד של מה קיים בענן (המשאבים עצמם עדיין קיימים ב-AWS, אבל Terraform "שוכח" אותם)
- קובץ ה-state עלול להכיל **מידע רגיש** (סיסמאות, keys) בטקסט גלוי – דיסק מקומי לא מאובטח מספיק לצוות

הפתרון: לשמור את קובץ ה-state לא אצלך במחשב, אלא **בענן**, במקום משותף ומאובטח, עם נעילה (lock) שמונעת משני אנשים לכתוב באותו רגע.

מה כל חלק בקובץ הזה בפועל יוצר ב-AWS

כשמריצים `terraform apply` על `main.tf` **בפעם הראשונה** (לפני הסרת ה- `#` מבלוק ה-backend), אנחנו בעצם יוצרים תשתית אמיתית בחשבון ה-AWS שלנו, בארבעה שלבים:

1. `aws_s3_bucket "terraform_state"` → יוצר **S3 bucket אמיתי**. זו התיקייה בענן שבה בעתיד יישמר קובץ ה-state.

- ⚠️ שמות S3 buckets הם **גלובליים בכל AWS** (לא רק בחשבון שלך) – לכן חובה לבחור שם ייחודי, אחרת תקבלו שגיאת `BucketAlreadyExists` .

2. `aws_s3_bucket_versioning` → מפעיל **גרסאות (versioning)** על ה-bucket. בכל פעם שקובץ ה-state ישתנה, S3 ישמור גם את הגרסה הקודמת – רשת ביטחון לשחזור אם הקובץ ייהרס או יתקלקל.
3. `aws_s3_bucket_server_side_encryption_configuration` → מצפין (AES256) את התוכן הנשמר בתוך ה-bucket, כי state עלול להכיל מידע רגיש.
4. `aws_dynamodb_table "terraform_locks"` → יוצר **טבלת DynamoDB אמיתית** למטרת **state locking**. כשמישהו מריץ `terraform apply` , Terraform "נועל" רשומה בטבלה לפני הכתיבה ל-state. אם מישהו אחר ינסה `apply` באותו רגע, הוא יקבל שגיאת "state is locked" במקום לגרום להתנגשות/קורפציה של הקובץ.

חשוב: בשלב הזה (לפני הסרת ה- `#`), קובץ ה-state של ה-config **עדיין מקומי, על הדיסק שלך**. יצרנו ב-AWS את "המחסן" (bucket) ואת "המנעול" (DynamoDB), אבל עדיין לא עברנו להשתמש בהם בפועל.

השלב השני: המעבר בפועל ל-remote

לאחר ההסרה של ה- `#` מהבלוק:

```
backend "s3" {
  bucket         = "<שם-שבחרת>"
  key            = "03-basics/import-bootstrap/terraform.tfstate"
  region         = "eu-north-1"
  dynamodb_table = "terraform-state-locking"
  encrypt        = true
}
```

והרצת `terraform init` שוב – **לא נוצר שום משאב חדש ב-AWS**. אנחנו רק אומרים ל-Terraform: "מעכשיו אל תשמור את קובץ ה-bookkeeping שלך על הדיסק המקומי – שמור אותו בתוך ה-bucket הזה, תחת הנתיב (`key`) הזה".

Terraform ישאל:

```
Do you want to copy existing state to the new backend?
Enter a value: yes
```

ולאחר אישור – הוא **מעתיק פיזית** את `terraform.tfstate` המקומי לתוך ה-S3 bucket, והתלות בקובץ המקומי מוסרת.

מהרגע הזה, כל `plan` / `apply` עתידי על ה-config הזה **קורא וכותב ישירות ל-S3**, לא לדיסק המקומי.

סיכום – שתי שכבות נפרדות שקל לבלבל ביניהן

	מה זה	איפה זה נשמר
המשאבים עצמם (bucket, DynamoDB table)	תשתית אמיתית ב-AWS שמשלמים עליה (כמעט חינם ב-free tier)	קיימים בענן, תמיד
קובץ ה-state של הקונפיג הזה	"פתק" שמתעד מה Terraform יצר	קודם על הדיסק המקומי → לאחר ה-`init` השני, בתוך ה-bucket שנוצר

זהו בדיוק ה"תרנגולת וביצה" (chicken and egg) של remote backends – אי אפשר לשמור state בענן לפני שיש איפה לשמור אותו, ולכן השלב הראשון תמיד חייב להיות local state.